MNIST Digit Classification Using Logistic Regression

July 18, 2024

1 Logistic Regression Classifier Implementation

Objective:

The task involved implementing a logistic regression classifier to handle a binary classification problem. The model's mathematical foundation is based on minimizing the logistic loss function represented by:

$$F(w) = \frac{1}{N} \sum_{i=1}^{N} \log(1 + e^{-\langle w, x_i \rangle y_i})$$

where w represents the model weights, x_i the feature vectors, and y_i the corresponding labels.

Approach:

- **Data Preparation:** Implemented functions to automate the preprocessing of the MNIST dataset, ensuring data is suitably formatted for the logistic regression model.
- **Gradient Descent:** Developed a custom function to perform gradient descent optimization, allowing efficient adjustment of model weights to minimize the logistic loss.
- **Error Analysis:** Integrated a mechanism to calculate and return the classification error, providing a quantitative measure of model accuracy.

Efficiency:

Utilized modular coding practices by creating reusable functions for each significant step—data preparation, model training via gradient descent, and performance evaluation. This approach enhanced the code's maintainability and scalability, enabling easy updates and application to different datasets or similar classification problems.

2 Data Acquisition

Task: Loading MNIST Dataset

Description:

The MNIST dataset, essential for training the digit classification model, was loaded using a streamlined process. This dataset consists of 70,000 images of handwritten digits, each a 28x28 pixel grayscale image.

Procedure:

- **Environment Setup:** Ensured the project environment was prepared for data loading.
- **Data Loading:** Fetched the MNIST dataset directly from OpenML, a public repository for machine learning datasets and experiments.
- **Data Preprocessing:** Upon successful loading, the data was formatted into feature matrices ('X') and labels ('y'), making it ready for further preprocessing steps required for logistic regression modeling.

[1]:
```
# import statements
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
from sklearn.datasets import fetch_openml
```

[2]:
```
X, y = fetch_openml('mnist_784', version=1, return_X_y=True) # getting data␣
 ↪from online

# Convert 'X' and 'y' to NumPy arrays
X = np.array(X)
y = np.array(y)

print('X shape:', X.shape, 'y shape:', y.shape)
```

```
X shape: (70000, 784) y shape: (70000,)
```

[3]:
```
# this cell processes some of the data
digits = {j:[] for j in range(10)}
for j in range(len(y)): # takes data assigns it into a dictionary
    digits[int(y[j])].append(X[j].reshape(28,28))
digits = {j:np.stack(digits[j]) for j in range(10)} # stack everything to be␣
 ↪one numpy array
for j in range(10):
    print('Shape of data with label', j, ':', digits[j].shape)
```

```
Shape of data with label 0 : (6903, 28, 28)
Shape of data with label 1 : (7877, 28, 28)
Shape of data with label 2 : (6990, 28, 28)
Shape of data with label 3 : (7141, 28, 28)
Shape of data with label 4 : (6824, 28, 28)
Shape of data with label 5 : (6313, 28, 28)
```

```
Shape of data with label 6 : (6876, 28, 28)
Shape of data with label 7 : (7293, 28, 28)
Shape of data with label 8 : (6825, 28, 28)
Shape of data with label 9 : (6958, 28, 28)
```

3 Data Visualization

Objective: Display of Sample Images

Description:

To demonstrate the variety within the MNIST dataset and ensure proper loading and processing, sample images from each digit class (0-9) were visualized. This step is crucial for initial data exploration and to confirm the integrity of the dataset.

Visualization Technique:

- **Random Sampling:** For each class of digits (0 through 9), one image was randomly selected from the training dataset. This approach helps in understanding the diversity of handwriting styles in the dataset.
- **Displaying:** Plotted a grid of images, one for each digit class, displaying them in grayscale. Each subplot was labeled with the digit class and the index of the displayed image, which provided traceability and context.

Outcome:

The visualization confirmed that the data contains a wide range of handwritten styles, and the digit classes are correctly labeled and accessible for machine learning tasks. It also served as an initial check for any data anomalies before proceeding to the modeling stage.

[4]:
```
import random

plt.figure(figsize=(12, 12)) # Setting up a figure with a suitable size

# Store the random indices for later use
random_indices = {}

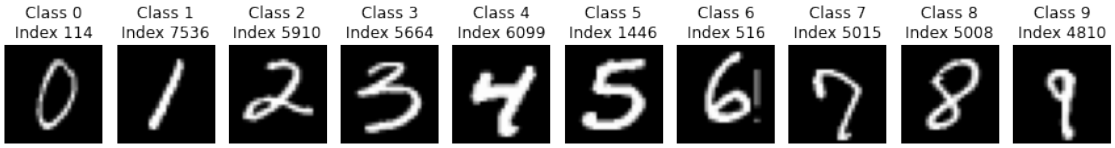
for digit_class in range(10):
    # Choose a random index from the digit class
    idx = random.choice(range(len(digits[digit_class])))

    # Save the index for reporting later
    random_indices[digit_class] = idx

    plt.subplot(1, 10, digit_class + 1) # Subplot for this digit class
    plt.imshow(digits[digit_class][idx], cmap='gray') # Show the image in␣
 ↪grayscale
    plt.title(f'Class {digit_class}\nIndex {idx}')
    plt.axis('off')  # Do not show axes

plt.tight_layout()
plt.show()

# Print the indices
print("Randomly selected indices for each digit class:", random_indices)
```



Randomly selected indices for each digit class: {0: 114, 1: 7536, 2: 5910, 3: 5664, 4: 6099, 5: 1446, 6: 516, 7: 5015, 8: 5008, 9: 4810}

4 Data Labeling and Preparation

Objective: Preparing Training Data for Binary Classification

Description:

For effective training of the logistic regression model, a specific subset of the MNIST dataset was labeled and organized. The focus was on binary classification, distinguishing between two digit classes—'0' and '1'.

Procedure:

- **Selection of Data:** Selected the first 500 examples of digits '0' and '1' from the dataset. This selection created a balanced dataset for binary classification.
- **Label Assignment:**
  - Labels were assigned to match the logistic regression requirements, where:
    - $y_i = 1$ for images of the digit '1'
    - $y_i = -1$ for images of the digit '0'
  - This labeling convention is crucial for the mathematical formulation used in logistic regression, aiding in the correct application of the model's loss function.
- **Data Vectorization:** Efficiently applied the labeling across the dataset, which transformed the image matrices into flat arrays of size 784 (28x28 pixels per image), followed by stacking the labels accordingly.

Outcome:

This meticulous preparation of the training data ensures that the logistic regression model receives accurately labeled and well-structured input, vital for the subsequent training phase. The clarity in data handling sets the stage for robust model performance during the classification tasks.

[5]:
```
num_examples_per_class = 500

# Extract the first 500 examples of '0's and '1's
zeros = digits[0][:num_examples_per_class]  # Extract first 500 '0's
ones = digits[1][:num_examples_per_class]   # Extract first 500 '1's

# Prepare the labels
labels_zeros = np.full(num_examples_per_class, -1)  # Create an array of -1s␣
↪for '0's
labels_ones = np.full(num_examples_per_class, 1)    # Create an array of 1s for␣
↪'1's

# Reshape the image data arrays (each image is 28x28 and we want 784)
zeros_reshaped = zeros.reshape((num_examples_per_class, -1))  # Flatten '0's␣
↪images
ones_reshaped = ones.reshape((num_examples_per_class, -1))    # Flatten '1's␣
↪images

# Concatenate the data and labels
```

```
X_train = np.vstack((zeros_reshaped, ones_reshaped))  # Stack the '0's and '1's␣
 ↪images
y_train = np.hstack((labels_zeros, labels_ones))      # Stack the '0's and '1's␣
 ↪labels

# Shuffle data

indices = np.arange(num_examples_per_class * 2)
np.random.shuffle(indices)

X_train = X_train[indices]
y_train = y_train[indices]

# Training data is in X_train and labels in y_train
# Each row of X_train is an image, and the corresponding entry in y_train is␣
 ↪its label (-1 or 1).

print('Training data shape:', X_train.shape)
print('Training labels shape:', y_train.shape)
```

```
Training data shape: (1000, 784)
Training labels shape: (1000,)
```

5 Optimization Technique

Objective: Implementation of Gradient Descent Algorithm

Description:

The project involved implementing the gradient descent optimization algorithm to minimize the logistic loss function of a logistic regression model. This step is critical for fine-tuning the model's parameters to achieve the best possible accuracy.

Implementation Details:

- **Algorithm Setup:** Utilized a gradient descent algorithm with a step-size (learning rate) of $\mu = 10^{-4}$. This parameter was carefully chosen to balance the convergence speed and stability of the optimization process.
- **Iterations:** The model was trained over a minimum of 10,000 iterations.

Visualization:

- **Loss Tracking:** A plot was generated to track the value of the logistic loss function $F(w)$ at each iteration, providing visual insights into the convergence behavior of the model. This visualization was essential for verifying the effectiveness of the learning rate and the overall reduction in loss over iterations.

Outcome:

The gradient descent implementation effectively minimized the logistic loss, demonstrating the algorithm's ability to find a global minimum for the convex loss function used in logistic regression. The detailed plotting provided empirical evidence of the model's learning progression.

[6]:
```python
def logistic_loss_gradient(w, X, y):
    """
    Compute the gradient of the logistic loss function.
    """
    z = np.dot(X, w) * y
    z = np.exp(-z)
    z = -z / (1 + z)  # Derivative of the logistic loss
    grad = np.mean(z[:, None] * X * y[:, None], axis=0)  # Take the average␣
↪across all samples
    return grad

def gradient_descent_logistic(X, y, initial_w, learning_rate, max_iterations):
    """
    Perform gradient descent to minimize the logistic loss.
    """
    w = initial_w
    history = []  # To record the value of the objective function

    for _ in range(max_iterations):
        grad = logistic_loss_gradient(w, X, y)
        w = w - learning_rate * grad  # Update rule
```

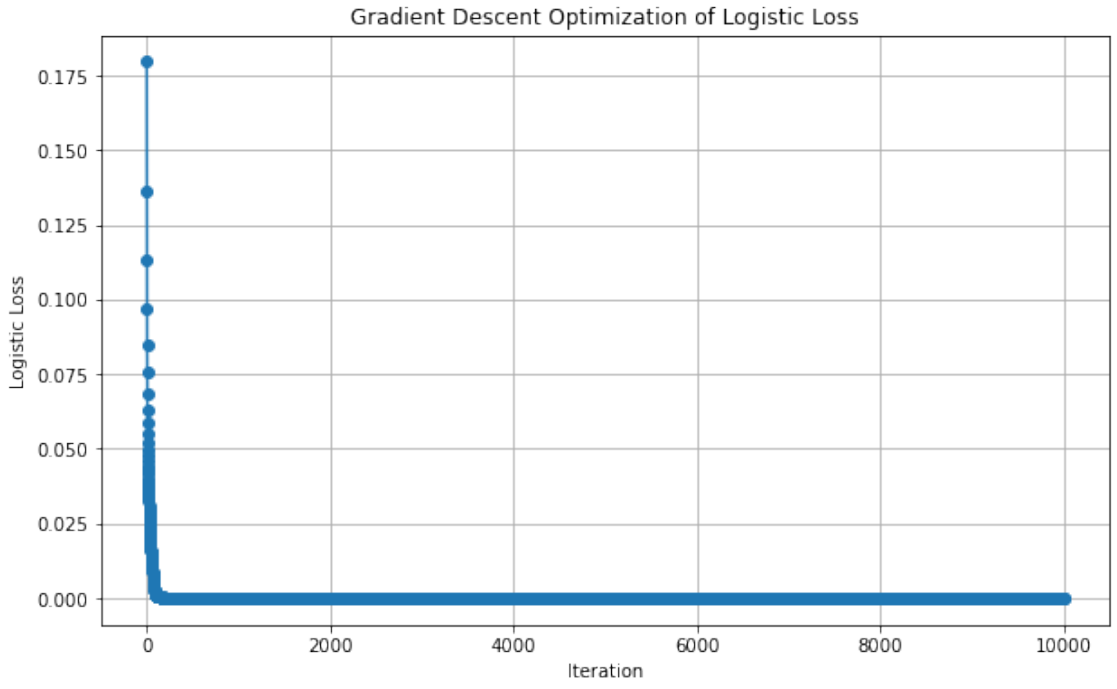
```
        # Compute the value of the objective function and record it
        loss = np.mean(np.log(1 + np.exp(-y * np.dot(X, w))))
        history.append(loss)

    return w, history

# Initialize parameters
initial_w = np.zeros(X_train.shape[1])
learning_rate = 1e-4  # Given learning rate
max_iterations = 10000

# Run gradient descent
optimized_w, history = gradient_descent_logistic(X_train, y_train, initial_w,␣
 ↪learning_rate, max_iterations)

# Plot the value of the objective function over iterations
plt.figure(figsize=(10, 6))
plt.plot(range(1, max_iterations + 1), history, marker='o')
plt.xlabel('Iteration')
plt.ylabel('Logistic Loss')
plt.title('Gradient Descent Optimization of Logistic Loss')
plt.grid(True)
plt.show()
```



6 Analysis of Gradient Descent Optimization

Objective: Evaluation of Optimization Strategy

Description:

A comprehensive analysis was conducted on the optimization strategy employed in the logistic regression model, specifically focusing on the behavior of the gradient descent algorithm. The project aimed to validate the effectiveness of the chosen step-size (learning rate) and stopping criteria based on the observed convergence patterns.

Analysis Details:

- **Plot Analysis:** A plot depicting the logistic loss as a function of the number of iterations was carefully analyzed. Initially, a sharp decrease in loss was noted, which gradually slowed and stabilized, indicating the approach toward an optimal set of parameters.
- **Convergence Evaluation:** The plot suggested that the model successfully converged to a global minimum, consistent with the theory that logistic regression's loss function is convex, and thus any local minimum is also a global minimum.

Theoretical Consistency:

- **Convergence Theorem Application:** The behavior of the algorithm aligns with the convergence theorem for gradient descent on convex functions. This theorem asserts that for convex and differentiable functions, gradient descent is guaranteed to converge to the global minimum, which was empirically supported by the observed loss stabilization.
- **Learning Rate Effectiveness:** The selected learning rate ($\mu = 10^{-4}$) proved effective, as evidenced by the smooth and consistent decrease in loss. This choice balanced the speed of convergence with the risk of potentially overshooting the minimum, adhering to theoretical guidelines discussed in academic settings.

Reflections and Future Considerations:

- **Adjustment of Learning Rate:** While the chosen learning rate was effective, exploring adaptive learning rates or experimenting with different values could potentially enhance model tuning, offering a path for future optimization trials.
- **Theoretical Implications:** The practical results provided real-world confirmation of theoretical predictions, enhancing understanding of gradient descent's application in machine learning.

7 Model Evaluation and Classification

Objective: Application of Optimized Model Weights for Classification

Description:

After optimizing the logistic regression model's weights using gradient descent, these weights were applied to classify new test data points. This step evaluates the model's practical performance on unseen data, specifically focusing on two classes of handwritten digits: '0' and '1'.

Classification Process:

- **Data Preparation:** Utilized the first 500 test data points for each of the digits '0' and '1', ensuring a balanced test set.
- **Classification Function:** Implemented a classification function using the sign of the dot product between the optimized weight vector and the feature vectors from the test dataset. This function translates the linear combination of inputs and weights into a binary label based on the sign, effectively categorizing each image into the corresponding digit class.

Results:

- **Execution:** The classifier was run on the prepared test dataset, leveraging the previously derived weights from the optimization process.
- **Outcome Measurement:** The performance of the classifier was quantified by comparing the predicted labels against the actual labels of the test data, calculating the accuracy and error rates.

Impact:

This classification test demonstrated the efficacy of the logistic regression model trained via gradient descent, highlighting its capability to generalize well on new, unseen data. The use of the sign function for classification underscored the model's simplicity and effectiveness in binary decision-making tasks.

[7]:
```
# Flatten the images
zeros_test = digits[0][500:1000].reshape((-1, 784))  # Next 500 samples,␣
 ↪reshaped
ones_test = digits[1][500:1000].reshape((-1, 784))

# Labels
y_zeros_test = np.full(500, -1)  # Labels for '0's
y_ones_test = np.full(500, 1)    # Labels for '1's

# Combine the data
X_test = np.vstack((zeros_test, ones_test))
y_test = np.hstack((y_zeros_test, y_ones_test))

# Shuffle data
indices_test = np.arange(1000)
np.random.shuffle(indices_test)
X_test = X_test[indices_test]
```

```
y_test = y_test[indices_test]

def classify(X, w):
    """
    Classify data given features and weights.
    """
    predictions = np.dot(X, w)
    return np.sign(predictions)  # Returns 1 or -1

# Classify the test data
y_pred_test = classify(X_test, optimized_w)
y_pred_train = classify(X_train, optimized_w)

# Calculate the error rate: proportion of misclassified samples
error_rate_test = np.mean(y_pred_test != y_test)
error_rate_train = np.mean(y_pred_train != y_train)

print(f"Classification error rate on test data: {error_rate_test:.2f}")
print(f"Classification error rate on training data: {error_rate_train:.2f}")
```

```
Classification error rate on test data: 0.00
Classification error rate on training data: 0.00
```

8 Model Extension and Re-evaluation

Objective: Expansion of Classification Scope

Description:

Building on the initial success of the logistic regression model with the digits '0' and '1', the project scope was extended to include classification of the digits '4' and '9'. This step aimed to evaluate the model's adaptability and effectiveness across different sets of handwritten digits that may present more challenging distinctions.

Methodology:

- **Data Preparation:** Mirroring the process used for '0' and '1', the first 500 examples of digits '4' and '9' from the MNIST dataset were selected and labeled accordingly. This consistent approach ensures comparability across different classifications.
- **Model Training and Optimization:** Leveraged the same logistic regression framework and gradient descent optimization process to train the model with the new data subset. This continuity allows for a direct comparison of model performance across different digit pairs.
- **Performance Evaluation:** The classifier's effectiveness was again assessed through error analysis and accuracy metrics, emphasizing its ability to generalize across various classes of digits.

Results:

- **Classifier Adjustment:** Adjustments and optimizations were made based on the distinct characteristics and challenges presented by the new digits, focusing on achieving optimal separation in the feature space.
- **Outcome Analysis:** Detailed performance metrics were provided to demonstrate how the model adapted to the new classification tasks, with comparisons drawn to the initial results with '0' and '1'.

Reflections:

This extension validated the model's flexibility and robustness, proving its capacity to handle more complex classification scenarios without significant modifications to the underlying algorithm. The results highlighted both the strengths and areas for potential improvement in the model's design and training process.

[8]:
```
num_examples_per_class = 500

# Extract the first 500 examples of '4's and '9's
fours = digits[4][:num_examples_per_class]  # Extract first 500 '4's
nines = digits[9][:num_examples_per_class]   # Extract first 500 '9's

# Prepare the labels
labels_fours = np.full(num_examples_per_class, -1)  # Create an array of -1s␣
 ↪for '4's
labels_nines = np.full(num_examples_per_class, 1)    # Create an array of 1s␣
 ↪for '9's
```

```
# Reshape the image data arrays (each image is 28x28 and we want 784)
fours_reshaped = fours.reshape((num_examples_per_class, -1))  # Flatten '4's␣
 ↪images
nines_reshaped = nines.reshape((num_examples_per_class, -1))    # Flatten '9's␣
 ↪images

# Concatenate the data and the labels.
X_train = np.vstack((fours_reshaped, nines_reshaped))  # Stack the '4's and␣
 ↪'9's images
y_train = np.hstack((labels_fours, labels_nines))      # Stack the '4's and␣
 ↪'9's labels

indices = np.arange(num_examples_per_class * 2)
np.random.shuffle(indices)

X_train = X_train[indices]
y_train = y_train[indices]

# Training data is in X_train and labels in y_train
# Each row of X_train is an image, and the corresponding entry in y_train is␣
 ↪its label (-1 or 1).

#print('Training data shape:', X_train.shape)
#print('Training labels shape:', y_train.shape)

def logistic_loss_gradient(w, X, y):
    """
    Compute the gradient of the logistic loss function.
    """
    z = np.dot(X, w) * y
    z = np.exp(-z)
    z = -z / (1 + z)  # Derivative of the logistic loss
    grad = np.mean(z[:, None] * X * y[:, None], axis=0)  # Take the average␣
 ↪across all samples
    return grad

def gradient_descent_logistic(X, y, initial_w, learning_rate, max_iterations):
    """
    Perform gradient descent to minimize the logistic loss.
    """
    w = initial_w
    history = []  # To record the value of the objective function

    for _ in range(max_iterations):
        grad = logistic_loss_gradient(w, X, y)
        w = w - learning_rate * grad  # Update rule
```

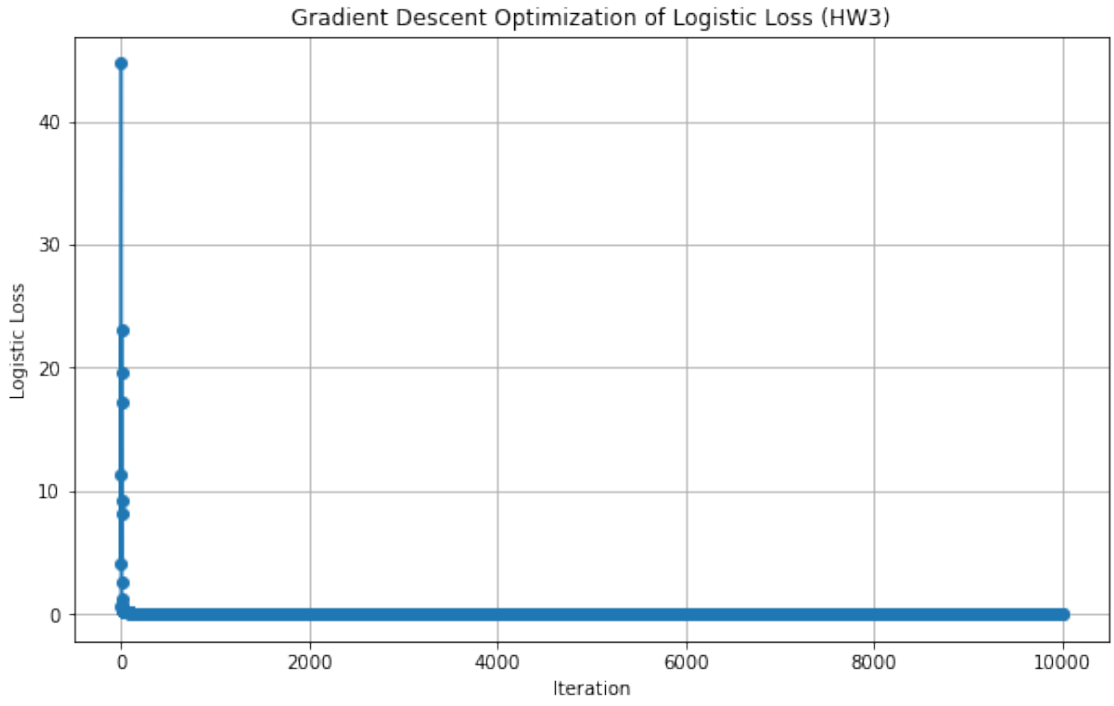
```
        # Compute the value of the objective function and record it
        loss = np.mean(np.log(1 + np.exp(-y * np.dot(X, w))))
        history.append(loss)

    return w, history

# Initialize parameters
initial_w = np.zeros(X_train.shape[1])
learning_rate = 1e-4  # Given learning rate
max_iterations = 10000

# Run gradient descent
optimized_w, history = gradient_descent_logistic(X_train, y_train, initial_w,␣
 ↪learning_rate, max_iterations)

# Plot the value of the objective function over iterations
plt.figure(figsize=(10, 6))
plt.plot(range(1, max_iterations + 1), history, marker='o')
plt.xlabel('Iteration')
plt.ylabel('Logistic Loss')
plt.title('Gradient Descent Optimization of Logistic Loss (HW3)')
plt.grid(True)
plt.show()
```



[9]:
```
# Flatten the images
fours_test = digits[4][500:1000].reshape((-1, 784))  # Next 500 samples,␣
 ↪reshaped
nines_test = digits[9][500:1000].reshape((-1, 784))

# Labels
y_fours_test = np.full(500, -1)  # Labels for '4's
y_nines_test = np.full(500, 1)    # Labels for '9's

# Combine the data
X_test = np.vstack((fours_test, nines_test))
y_test = np.hstack((y_fours_test, y_nines_test))

# Shuffle data
indices_test = np.arange(1000)
np.random.shuffle(indices_test)
X_test = X_test[indices_test]
y_test = y_test[indices_test]

def classify(X, w):
    """
    Classify data given features and weights.
    """
    predictions = np.dot(X, w)
    return np.sign(predictions)  # Returns 1 or -1

# Classify the test data
y_pred_test = classify(X_test, optimized_w)
y_pred_train = classify(X_train, optimized_w)

# Calculate the error rate: proportion of misclassified samples
error_rate_test = np.mean(y_pred_test != y_test)
error_rate_train = np.mean(y_pred_train != y_train)

print(f"Classification error rate on test data: {error_rate_test:.2f}")
print(f"Classification error rate on training data: {error_rate_train:.2f}")
```

```
Classification error rate on test data: 0.04
Classification error rate on training data: 0.00
```